

✓ Lab 2 — Exercise: AI Tools & the Data Mining Pipeline

ITCS 3162 — Introduction to Data Mining

Name: *your name here* **Date:** *today's date*

Complete the exercises below. **Some questions ask you to use an AI assistant** — when you do, you must:

1. Paste the prompt you used in the markdown cell labeled "PROMPT".
2. Paste the AI's response (or key parts) in the cell labeled "AI RESPONSE".
3. Write your own evaluation in "YOUR EVALUATION".

This is the appropriate-use pattern: prompt, response, your judgment.

Exercise 1 — Identify appropriate use (15 pts)

For each scenario, decide whether the AI use is **appropriate** or **inappropriate** and explain why in one sentence.

#	Scenario
A	A student doesn't understand what "stratified sampling" means and asks an AI to explain it with an example.
B	A student pastes the full text of an exercise question and submits the AI's answer as their reflection.
C	A student has a <code>KeyError</code> they can't figure out and asks the AI what it means.
D	A student asks the AI to write all their markdown answers in this notebook so they can focus on the code.
E	A student writes their own code, then asks the AI to suggest more idiomatic alternatives, then chooses one and explains it in their own words.

YOUR ANSWERS:

- **A:** Appropriate. Using AI to understand something.
- **B:** Inappropriate. plagiarizing from AI.
- **C:** Appropriate. Using AI to troubleshoot
- **D:** Inappropriate. Cheating.
- **E:** Appropriate. Using AI for suggestions.

Exercise 2 — Rewrite a weak prompt (35 pts)

Below is a weak prompt. Rewrite it so it's specific, scoped, and verifiable. Imagine you have a DataFrame `(df)` with a column `(price)` (numeric) and you want to detect rows whose price is unusually high.

Weak prompt: "find weird stuff"

YOUR REWRITTEN PROMPT:

find unusually high prices in this table.

✓ Exercise 3 — Prompt, response, evaluate (20 pts)

Use any AI assistant (ChatGPT, Claude, Copilot chat, Gemini, etc.) to answer this question:

"In scikit-learn, what is the difference between `fit`, `transform`, and `fit_transform`? When would I use each?"

Fill in all three cells below.

PROMPT (what you actually typed): In scikit-learn, what is the difference between `fit`, `transform`, and `fit_transform`? When would I use each?

AI RESPONSE (paste it here, or summarize the key points):

In scikit-learn, these methods are mainly used with preprocessors like scalers, encoders, imputers, and dimensionality reduction tools.

`fit()`

Learns information from the data.

It does not change the data yet. It just calculates whatever statistics are needed.

Examples:

Mean and standard deviation for scaling Categories for one-hot encoding Principal components for PCA from sklearn.preprocessing
import StandardScaler

```
scaler = StandardScaler() scaler.fit(X_train)
```

After this:

```
scaler.mean_ scaler.scale_
```

have been learned from X_train.

Use fit() when:

You want to learn parameters first You may transform data later You are separating training and testing workflows carefully transform()

Uses the learned information to modify data.

```
X_train_scaled = scaler.transform(X_train)
```

This applies:

$$z = \frac{x - \mu}{\sigma}$$

but does not relearn anything.

You also use transform() on test data:

```
X_test_scaled = scaler.transform(X_test)
```

This is important because the test set must use the same statistics learned from training data.

Use transform() when:

The object has already been fitted You want to apply the same transformation to new data fit_transform()

Combines both steps:

Learn parameters Immediately transform the data X_train_scaled = scaler.fit_transform(X_train)

Equivalent to:

```
scaler.fit(X_train) X_train_scaled = scaler.transform(X_train)
```

Use fit_transform() when:

Working with training data You want convenience and efficiency Typical Workflow from sklearn.preprocessing import StandardScaler

```
scaler = StandardScaler()
```

✓ Learn from training data and transform it

```
X_train_scaled = scaler.fit_transform(X_train)
```

Only transform test data

```
X_test_scaled = scaler.transform(X_test)
```

This avoids data leakage.

Quick Analogy

Think of a teacher grading tests:

fit() = teacher studies the grading rubric transform() = teacher grades papers using that rubric fit_transform() = teacher learns rubric and grades immediately Common Objects That Use These Methods StandardScaler MinMaxScaler OneHotEncoder SimpleImputer PCA

Example with PCA:

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
```

```
X_reduced = pca.fit_transform(X_train)
```

Here PCA:

Learns the important directions Projects data into 2 dimensions Important Rule

For machine learning pipelines:

Training data → fit_transform() Validation/test/new data → transform()

Never fit() on test data.

YOUR EVALUATION — answer all three:

1. Was the response correct? How did you verify?
2. Was anything missing or unclear?
3. In your own words (no copy-paste), explain the difference between `fit` and `transform`.

YOUR ANSWER:

I know the response was correct as I have used these different methods in other work. Nothing was unclear but I feel like the answer was a bit too long. fit learns/trains wit the data and transform uses what was learned in training to modify data.

Exercise 4 — Map the pipeline (15 pts)

Below is a list of activities. For each, write which **pipeline stage** it belongs to (Problem understanding / Load / Explore / Preprocess / Model / Evaluate).

#	Activity	Stage
A	Running <code>df.describe()</code> to see column means and quartiles	
B	Choosing whether to use a decision tree or logistic regression	
C	Replacing missing ages with the median age	
D	Talking to a stakeholder to clarify what "customer churn" means	
E	Computing accuracy and confusion matrix on the held-out test set	
F	Reading a CSV with <code>pd.read_csv("data.csv")</code>	
G	One-hot encoding a <code>country</code> column	
H	Plotting a histogram of <code>salary</code>	

```
# Exercise 5 — Build your own mini-pipeline (20 pts)
#
# Using the wine dataset (built into sklearn), build a complete pipeline that:
# 1. Loads the data as a DataFrame
# 2. Prints the shape and the class distribution of the target
# 3. Splits into 70/30 train/test with random_state=42, stratified on the target
# 4. Builds a sklearn Pipeline with StandardScaler + LogisticRegression(max_iter=500)
# 5. Fits on the training data and reports test accuracy
#
# Fill in the TODOs below.

import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 1. LOAD
wine = load_wine(as_frame=True)
df = wine.frame
# TODO: print df.shape
# TODO: print df["target"].value_counts()
print(df.shape)
print(df["target"].value_count())

# 2. SPLIT
X = df.drop(columns="target")
y = df["target"]
# TODO: train_test_split with test_size=0.30, random_state=42, stratify=y
# X_train, X_test, y_train, y_test = ...
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# 3. PIPELINE
# TODO: build the Pipeline with two steps named "scale" and "model"
# pipe = Pipeline([...])
```

```
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=500))
])

# 4. FIT + SCORE
# TODO: fit on the training data
# TODO: print the test accuracy with 3 decimal places
pipe.fit(X_train, y_train)

accuracy = pipe.score(X_train, y_train)
print(f"Accuracy: {accuracy: .3f}")
```

T **B** **I** **<>** **↺** **↻** **🔍** **🔗** **📄** **📌** **📋** **📏** **🔧** **😊** **⋮** **Close**

Exercise 6 – Reflection (15 pts)

In 4–6 sentences, answer **both**:

1. What's one task in this course you think AI tools would help you most, and one task where you'd intentionally not use them? Why?
2. Looking at the six-stage pipeline, which stage do you expect to find hardest and why?

YOUR ANSWER:

I think one task in this course AI would help me with is when I would need to debug code. One place I would not use AI is for assignments such as zybooks. Using AI for these types of tasks is not only unethical, it prevents me from learning. I think the hardest stage is the data preparation stage. Cleaning data is very complicated and time consuming.

Exercise 6 — Reflection (15 pts)

In 4–6 sentences, answer **both**:

1. What's one task in this course you think AI tools would help you with the most, and one task where you'd intentionally not use them? Why?
2. Looking at the six-stage pipeline, which stage do you expect to find hardest and why?

YOUR ANSWER: I think one task in this course AI would help me with is when I would need to debug code. One place I would not use AI is for assignments such as quizzes or zybooks. Using AI for these types of tasks is not only unethical, but it also prevents me from learning. I think the hardest stage is the data preparation stage. Cleaning data is very complicated and time consuming.

Submission checklist

- ☐ Name and date filled in at the top
- ☐ All **YOUR ANSWER** / **TODO** prompts completed
- ☐ Exercise 3 includes a PROMPT, AI RESPONSE, and YOUR EVALUATION
- ☐ Code cell in Exercise 5 runs and prints test accuracy
- ☐ **Restart & Run All** completes without errors
- ☐ Downloaded as **.ipynb** and submitted via Canvas